SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:100**

Annexure A

EXPERIMENTS

Code of Conduct:

I P.Karpaga shree(192424042) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
P.Karpaga shree

Annexure A

EXPERIMENTS

Q1.You are developing a compiler-based desk calculator that evaluates arithmetic expressions using syntax-directed definitions (SDD).

The system should:

(i) Accept an arithmetic expression with operators +, -, *, / and parentheses

(ii) Use syntax-directed evaluation to compute values based on operator precedence

(iii) Optimize evaluation by reducing unnecessary temporary computations

(iv) Display the final result of the expression

Demonstrate how SDD rules (synthesized attributes) are used for expression evaluation.

SDD

Grammar with semantic rules:

E → E + T	{ E.val = E1.val + T.val }
E → E – T	{ E.val = E1.val – T.val }
E → T	{ E.val = T.val }
T → T * F	{ T.val = T1.val * F.val }
T → T / F	{ T.val = T1.val / F.val }
T → F	{ T.val = F.val }
F → (E)	{ F.val = E.val }
F → digit	{ F.val = digit }

Q2. In a compiler, postfix expressions are evaluated during code generation using stack-based bottom-up evaluation.

Write a C program that:

(i) Evaluates a postfix expression

(ii) Demonstrates S-attributed definition evaluation

(iii) Shows how intermediate results are computed

Q3.You are designing a parser that uses L-attributed definitions, where inherited attributes are passed from parent to child.

Write a C program that:

(i).Evaluates a simple expression like `a + b * c`

(ii) Uses function parameters to simulate inherited attributes

(iii). Ensures correct precedence handling

(iv). Demonstrates top-down evaluation logic

Explain how inherited attributes influence computation.

Q4.In a compiler, it is important to check whether two type expressions are equivalent.

Write a C program that:

(i).Accepts two type expressions (e.g., `int`, `float`, `int*`)

(ii).Compares them for **type equivalence**

(iii).Displays whether they are:

(a).Equivalent

(b).Not equivalent

Extend the logic to handle pointer types (`int*`, `float*`).

Q5.You are building a semantic analyzer that detects type errors in arithmetic expressions.

Write a C program that:

(i). Accepts two operands and their types

(ii). Checks whether the operation (e.g., `+`, `*`) is valid

(iii). Reports:

(a).Valid expression

(b).Type error (e.g., `int + char*`)

Demonstrate how compilers detect semantic errors during type checking

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

CO3 AT1 – Experiments

1) Syntax Directed Definition (SDD)

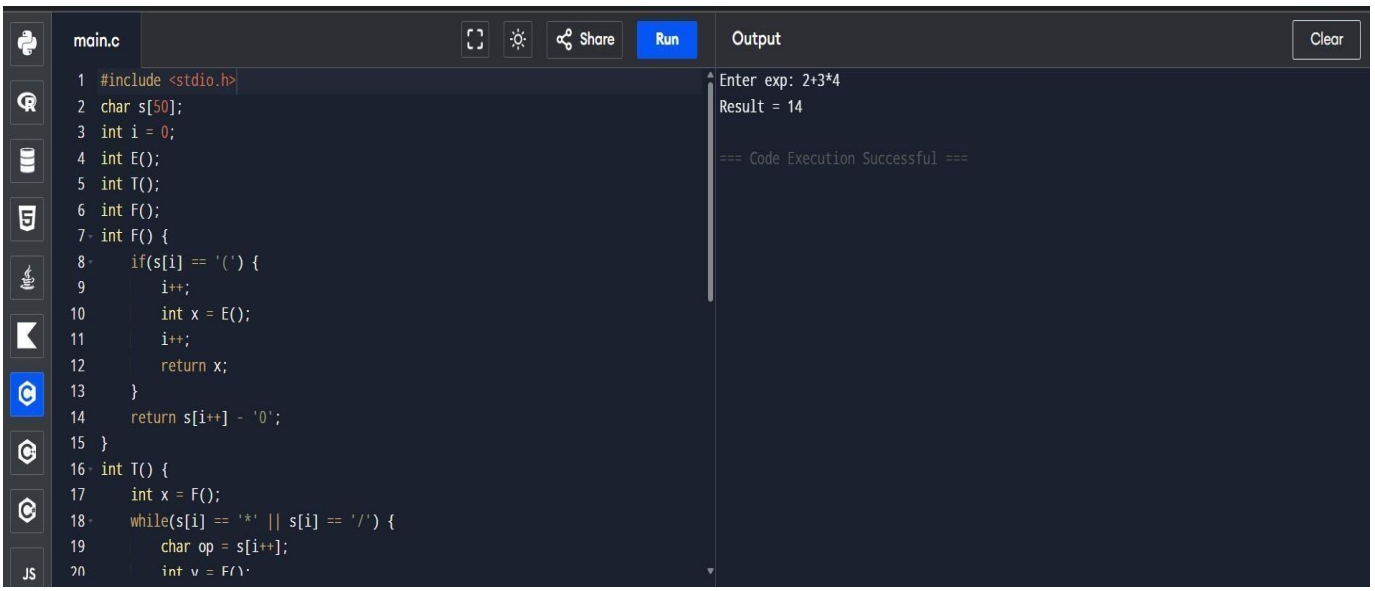
Aim

To evaluate arithmetic expressions using Syntax Directed Definitions (SDD) with synthesized attributes.

Procedure

1. Define grammar for expressions (E, T, F).
2. Assign synthesized attribute val to each symbol.
3. Apply rules based on operator precedence.
4. Evaluate expression step by step.
5. Display final computed value.

Output



2) Postfix Evaluation

(S-Attributed) Aim

To evaluate postfix expression using stack and S-attributed definition.

Procedure

1. Read postfix expression.
2. Use stack for evaluation.
3. Push operands into stack.
4. Pop operands when operator appears.
5. Perform operation and push result.
6. Final stack value is result.

Output

3) L-Attributed

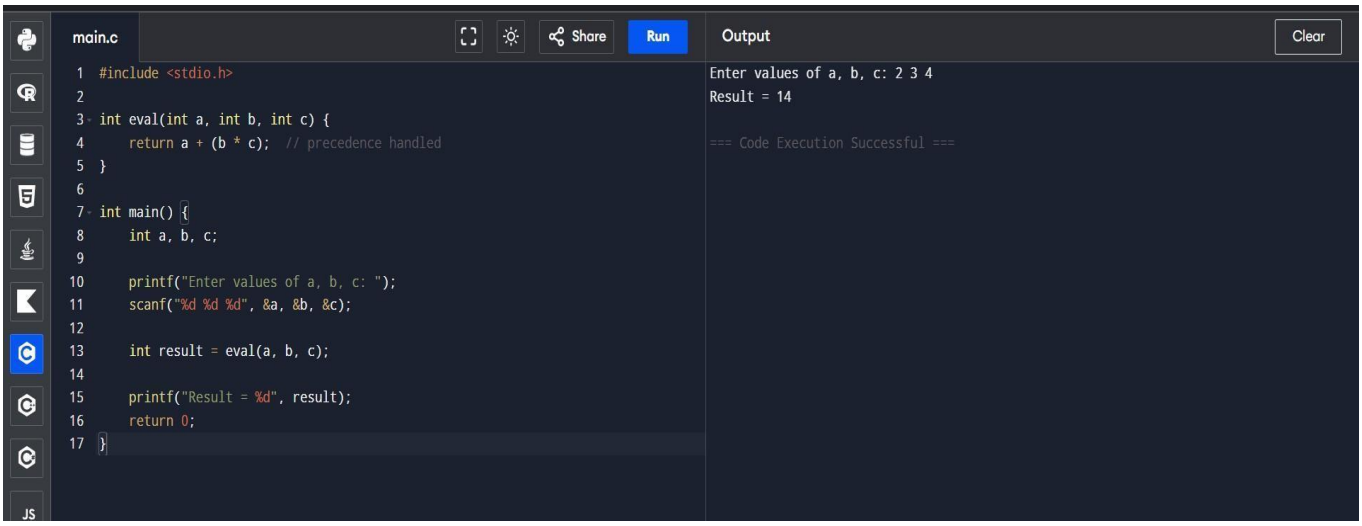
Definition Aim

To evaluate expressions using L-attributed definitions with inherited attributes.

Procedure

1. Use function parameters to pass inherited values.
2. Maintain operator precedence.
3. Evaluate expression in top-down manner.
4. Return computed result.

Output



4) Type

Equivalence Aim

To check whether two type expressions are equivalent.

Procedure

1. Accept two type strings.
2. Compare both strings.
3. Check pointer levels (*).
4. Display result.

Output

main.c

```c
#include <stdio.h>
#include <string.h>

int main() {
    char t1[20], t2[20];

    printf("Enter type 1: ");
    scanf("%s", t1);
    printf("Enter type 2: ");
    scanf("%s", t2);

    if(strcmp(t1, t2) == 0)
        printf("Equivalent Types\n");
    else
        printf("Not Equivalent Types\n");

    return 0;
}
```

Output

```
Enter type 1: int*
Enter type 2: int*
Equivalent Types


=== Code Execution Successful ===
```

5) Type Checking (Semantic Analysis) Aim

To detect type errors in arithmetic expressions.

Procedure

1. Accept operand types.
2. Check compatibility.
3. Allow valid operations.
4. Display error for invalid types.

Output

main.c

```c
int main() {
    char t1[20], t2[20], op;

    printf("Enter type of operand1: ");
    scanf("%s", t1);
    printf("Enter operator (+, *): ");
    scanf(" %c", &op);
    printf("Enter type of operand2: ");
    scanf("%s", t2);

    if((strcmp(t1, "int") == 0 || strcmp(t1, "float") == 0) &&
       (strcmp(t2, "int") == 0 || strcmp(t2, "float") == 0)) {
        printf("Valid Expression\n");
    } else {
        printf("Type Error\n");
    }

    return 0;
}
```

Output

```
Enter type of operand1: int + float
Enter operator (+, *): Enter type of operand2: Valid Expression


=== Code Execution Successful ===
```